INSTITUTO MILITAR DE ENGENHARIA (IME)

Seção de Engenharia Elétrica — SE/3

Programação Aplicada à Engenharia Elétrica (SE/3 - Eletrônica)

Guia Mestre de Orientação e Desenvolvimento: Passos 1 e 2

Modelagem POO em C++, Subsistema IIO no Linux, Compilação Cruzada e Deploy Embarcado

Instituição:	Instituto Militar de Engenharia (IME) / SE/3
Projeto Final:	Monitoramento Inteligente de Carga Sensível em Operações Logísticas
Kit de Desenvolvimento:	STMicroelectronics **STM32MP157D-DK1** (Linux Embarcado OpenSTLinux)
Sensor do Grupo (Nº 3):	Módulo de Obstáculo Infravermelho Reflexivo Digital (**HW-201** / LM393)
Passo 1 (Entrega 1):	**26/10/2026 às 23:59** — Classe C++ com Doxygen no Git (2,0 pts / 800 pts)
Passo 2 (Entrega 2):	**02/11/2026 às 23:59** — Binário ARM na Placa + README com Prints (2,0 pts / 800 pts)

1 Cronograma Oficial e Metas Imediatas

Com a conclusão e aprovação da Entrega 0 (montagem física e conferência da ligação segura em 3,3 V), o projeto entra na sua fase central de desenvolvimento de software de tempo real e sistemas embarcados. A tabela abaixo sintetiza todo o ciclo avaliativo:

Etapa	Prazo Final	Pontos	Entregável Principal
Entrega 0	19/10/2026	Obrig.	Validação física da montagem com o kit desligado (Concluído).
Entrega 1	**26/10/2026**	2,0 pts	Classe C++ (`IrObstacleSensor`) documentada com Doxygen no GitHub.
Entrega 2	**02/11/2026**	2,0 pts	Executável compilado de forma cruzada rodando no Linux da placa.
Entrega 3	09/11/2026	2,0 pts	Envio contínuo das leituras via socket UDP pela interface Ethernet.
Entrega 4	16/11/2026	2,0 pts	Interface gráfica (GUI) no computador central com histórico e log CSV.
Entrega 5	16/11/2026	2,0 pts	**Mesma data da Entrega 4!** Vídeo, documentação final e integração.

A Regra Crítica de POO da Disciplina (Critério Eliminatório)

O edital oficial estabelece de forma categórica: **é terminantemente proibida a utilização de variáveis globais e de funções soltas fora de classes**.

- **Por que essa regra existe?** Na Entrega 5, todos os módulos desenvolvidos pelos grupos da turma serão integrados em uma central de monitoramento. Se dois alunos criarem uma variável chamada `int valor;` ou uma função `void ler();`, ocorrerá colisão de símbolos no momento da ligação (*linker error*).
- **A solução correta:** Todo estado (caminho de arquivos, limiares, histórico) deve ser atributo `private` da classe, acessível apenas por métodos `public` devidamente documentados.

2 MÓDULO 1 (Passo 1): Modelagem C++, Linux IIO e Doxygen

2.1 A Teoria: Como o Linux Embarcado Trata os Pinos Físicos?

A placa STM32MP157D-DK1 não roda o código em modo microcontrolador puro (*bare-metal*). Ela executa uma distribuição Linux completa baseada em Buildroot/OpenSTLinux. No ambiente Linux, vigora o paradigma universal: **"Tudo é um arquivo" (*Everything is a file*)**.

O Subsistema IIO (Industrial I/O)

Para permitir que programas em espaço de usuário (*user space*) acessem conversores Analógico-Digitais (ADC), acelerômetros e sensores industriais de forma padronizada e segura, o Kernel do Linux expõe o subsistema **IIO** através do sistema de arquivos virtual `sysfs`:

`/sys/bus/iio/devices/iio:device0/`

Dentro desse diretório virtual, o driver de hardware cria arquivos como `in_voltage0_raw`, `in_voltage1_raw`, etc. Quando o seu programa em C++ abre esse arquivo e lê o seu conteúdo, o Kernel realiza a conversão A/D instantânea no pino físico da placa e devolve o valor como um número inteiro em formato texto (por exemplo, `0` para 0 V ou `4095`/`65535` para 3,3 V).

2.2 Passo Presencial (Laboratório): Descobrir o Canal Exato do ADC

Na Entrega 0, o fio de sinal do seu sensor foi conectado no **Pino 5 do conector de 6 furos (ADC)**. Agora, no laboratório, você precisa descobrir qual número `X` corresponde a esse pino físico.

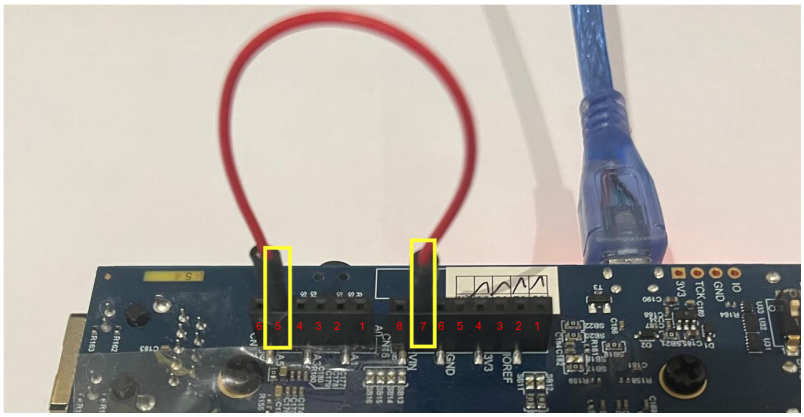


Figura 1: Procedimento oficial de descoberta de canal: Jumper ligando diretamente o Pino 5 do ADC ao GND (Pino 6/7) para forçar leitura zero no canal correspondente.

Procedimento de 5 Minutos no Laboratório

1. **Aterrar o pino de leitura:** Com a placa desligada, retire temporariamente o fio do sensor do Pino 5 do bloco ADC e coloque um jumper conectando o **Pino 5** diretamente ao **Pino 7 (GND)** do bloco de alimentação.
2. **Energizar e conectar a Serial:**
   - Conecte o cabo USB-C na porta `PWR_IN`.
   - Conecte o cabo Micro-USB na porta serial `ST-LINK` e abra o TeraTerm/PuTTY configurado para **115200 8N1**.
   - Pressione Enter e efetue login com usuário `root` e senha `root`.
3. **Consultar os canais IIO no terminal:**

```
cd /sys/bus/iio/devices/iio:device0/
cat in_voltage*_raw
```

4. **Identificação:** O arquivo `in_voltageX_raw` que retornar o valor **0** (ou ruído elétrico quase nulo, ex.: 1 ou 2) é o canal exato do seu sensor!

5. **Teste com o Sensor HW-201:** Desligue a placa, retire o jumper de aterramento e reconecte os 3 fios do sensor HW-201. Ao ligar e ler o arquivo identificado:

   - Com o compartimento livre (sem reflexão): leitura alta (nível lógico ALTO $\approx$ 3,3 V).
   - Com a tampa fechada/mão próxima: o feixe reflete, a saída vai para nível BAIXO e a leitura cai para valores próximos de zero.

2.3 Passo em Casa: Simulando o Hardware no seu Computador

Você não precisa da placa física para desenvolver, depurar e testar 100% da sua classe C++. Como a leitura ocorre através de um arquivo de texto, você pode criar um simulador local no seu computador (Linux, WSL ou terminal Git Bash):

Criando o Ambiente de Teste Caseiro

```
# 1. Cria diretorio simulando a estrutura sysfs do Linux
mkdir -p /tmp/mock_iio

# 2. Simula o estado NORMAL (tampa fechada / feixe refletindo -> 0V)
echo "150" > /tmp/mock_iio/in_voltage0_raw

# 3. Simula o estado de VIOLACAO (tampa aberta / sem reflexao -> 3.3V)
echo "4050" > /tmp/mock_iio/in_voltage0_raw
```

Projetando a sua classe para aceitar o caminho do arquivo no construtor (com valor padrão apontando para a placa real), o mesmo código compilará e executará tanto no seu PC em casa quanto na placa no laboratório!

2.4 Modelando a Classe C++ (`IrObstacleSensor`)

A estrutura deve seguir estritamente o padrão da engenharia de software moderna em C++: divisão entre o contrato de interface (`include/IrObstacleSensor.hpp`) e a implementação dos métodos (`src/IrObstacleSensor.cpp`).

Estrutura do Cabeçalho (`include/IrObstacleSensor.hpp`)

```
#ifndef IR_OBSTACLE_SENSOR_HPP
#define IR_OBSTACLE_SENSOR_HPP

#include <string>

/**
 * @file IrObstacleSensor.hpp
 * @brief Classe de aquisicao e interpretacao do Sensor de Obstaculo IR
     HW-201.
 */
class IrObstacleSensor {
private:
```

```
    std::string devicePath;  /**< Caminho do arquivo IIO no sysfs ou
       arquivo mock */
    int rawThreshold;        /**< Limiar de comutacao entre estado
       normal e violacao */
    int lastRawValue;        /**< Ultima leitura numerica bruta obtida
       do conversor ADC */
    double referenceVoltage; /**< Tensao de referencia do circuito (3.3
        V) */
    int maxAdcResolution;    /**< Resolucao maxima da escala do
       conversor (ex: 4095) */

public:
    /**
     * @brief Construtor da classe com parametros padrao configuraveis.
     * @param path Caminho do arquivo sysfs IIO ou arquivo mock para
        testes em casa.
     * @param threshold Limiar de decisao logica entre nivel alto e
        baixo.
     * @param vref Tensao de alimentacao de referencia em Volts (3.3V).
     * @param maxAdc Escala maxima do conversor A/D (4095 para 12 bits)
        .
     */
    explicit IrObstacleSensor(
        const std::string& path = "/sys/bus/iio/devices/iio:device0/
           in_voltage0_raw",
        int threshold = 2000,
        double vref = 3.3,
        int maxAdc = 4095
    );

    int readRaw();                      /**< Realiza a leitura atomica
       do arquivo IIO */
    double getVoltage() const;          /**< Converte a leitura bruta em
        tensao (Volts) */
    bool isObstacleDetected() const;    /**< Retorna true se houver
       reflexao (Active-Low) */
    bool isCompartmentViolated() const;/**< Avalia a integridade da
       carga monitorada */
    int getLastRawValue() const;        /**< Retorna o ultimo valor lido
        */
    void setThreshold(int newThreshold);/**< Ajuste dinamico do limiar
       de alerta */
};

#endif // IR_OBSTACLE_SENSOR_HPP
```

Implementação dos Métodos (`src/IrObstacleSensor.cpp`) e Princípio RAII

Ao manipular arquivos no Linux com C++, a biblioteca padrão `<fstream>` aplica o conceito de **RAII** (***Resource Acquisition Is Initialization***):

```
#include "IrObstacleSensor.hpp"
#include <fstream>
#include <stdexcept>

IrObstacleSensor::IrObstacleSensor(const std::string& path, int
   threshold, double vref, int maxAdc)
```

```
    : devicePath(path), rawThreshold(threshold), lastRawValue(0),
      referenceVoltage(vref), maxAdcResolution(maxAdc) {}

int IrObstacleSensor::readRaw() {
    std::ifstream file(devicePath);
    if (!file.is_open()) {
        throw std::runtime_error("Falha ao abrir canal IIO no caminho:
           " + devicePath);
    }
    file >> lastRawValue;
    return lastRawValue; // O arquivo eh fechado automaticamente pelo
       destrutor de ifstream!
}

bool IrObstacleSensor::isObstacleDetected() const {
    // Sensor HW-201 com comparador LM393 opera em Active-Low:
    // Reflexao detectada (objeto proximo) -> Saida comuta para nivel
       BAIXO (0V / ADC proximo a 0)
    return lastRawValue < rawThreshold;
}

bool IrObstacleSensor::isCompartmentViolated() const {
    // No contexto da carga: se o feixe nao reflete na tampa, o
       compartimento foi violado!
    return !isObstacleDetected();
}
```

2.5 Documentação com Doxygen e Submissão da Entrega 1

O edital exige documentação completa da classe via Doxygen:

1. **Tags fundamentais:** `@file` (descrição do arquivo), `@class` (escopo da classe), `@brief` (resumo sucinto do método), `@param` (significado dos parâmetros recebidos) e `@return` (valor de retorno).
2. **Geração do Doxyfile:** Execute `doxygen -g Doxyfile` no terminal. Edite as linhas:
   - `INPUT = include src`
   - `EXTRACT_PRIVATE = YES` (para documentar também os atributos privados).
3. **Geração e Validação:** Execute `doxygen Doxyfile` e abra `html/index.html` no navegador.
4. **Envio do Link do Commit:** Faça o commit no Git, envie com `git push origin main`, abra o GitHub no navegador, clique na aba **Commits**, selecione o commit gerado e envie a URL direta no Google Classroom!

3 MÓDULO 2 (Passo 2): Compilação Cruzada e Deploy Embarcado

A **Entrega 2 (02/11/2026)** consiste em gerar o executável final funcional, rodando diretamente no Linux embarcado da placa STM32MP157D-DK1, lendo o sensor físico e com instruções e prints devidamente documentados no `README.md`.

3.1 Fundamentos de Compilação Cruzada (*Cross-Compilation*)

Por que não podemos compilar com o g++ comum do PC?

O processador do seu computador é da arquitetura **x86_64 (Intel/AMD)**, com um conjunto de instruções específico. Já o processador da sua placa STM32MP157D-DK1 é um **Dual-core Arm Cortex-A7 (ARMv7-A de 32 bits)** com ponto flutuante via hardware (`vfpv4-d16 / neon`).

- Se você compilar no PC com o `g++` nativo, o binário conterá instruções x86_64 que o processador ARM da placa não é capaz de decodificar (resultando em erro imediato de execução: *"cannot execute binary file: Exec format error"*).
- Para resolver isso, utilizamos uma **Toolchain de Compilação Cruzada**, na qual o compilador roda no seu computador x86_64 (*Host*), mas traduz a sintaxe C++ gerando código de máquina ARM (*Target*).

3.2 Configuração da Toolchain no Linux

O projeto disponibiliza duas opções de ambiente de compilação cruzada no repositório:

1. **Opção A: Toolchain Buildroot** (`arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz`):

```
# Extrair o pacote da toolchain no seu diretorio de desenvolvimento
tar -xvf arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz

# Localizar o compilador g++ dentro da pasta extraida:
export PATH=$PATH:/caminho/da/toolchain/bin
arm-buildroot-linux-gnueabihf-g++ --version
```

2. **Opção B: SDK Yocto OpenSTLinux** (`SDK-x86_64-stm32mp1-...`):
   Permite inicializar as variáveis de ambiente completas com suporte ao *sysroot* da placa.

3.3 Automação Profissional com `Makefile`

Para garantir reprodutibilidade e atender às exigências de compilação documentadas no edital, crie um `Makefile` na raiz do projeto com alvos bem definidos:

Estrutura do `Makefile` para Compilação Híbrida (PC e ARM)

```
# Compilador nativo (para testes locais no PC em casa)
CXX ?= g++
CXXFLAGS = -Wall -Wextra -std=c++17 -Iinclude -O2

# Toolchain de Cross-Compilacao (para o processador ARM da STM32MP1)
CROSS_PREFIX ?= arm-buildroot-linux-gnueabihf-
CROSS_CXX = $(CROSS_PREFIX)g++

SRCS = src/IrObstacleSensor.cpp src/main.cpp
TARGET_NATIVE = app_sensor
TARGET_ARM = app_sensor_arm
```

```
.PHONY: all cross clean doc

all: $(TARGET_NATIVE)

$(TARGET_NATIVE): $(SRCS)
        $(CXX) $(CXXFLAGS) -o $@ $^
        @echo "Binario nativo para PC gerado com sucesso!"

cross: $(SRCS)
        $(CROSS_CXX) $(CXXFLAGS) -o $(TARGET_ARM) $^
        @echo "Binario cross-compilado para ARM Cortex-A7 gerado com
            sucesso!"

doc:
        doxygen Doxyfile

clean:
        rm -f $(TARGET_NATIVE) $(TARGET_ARM) *.o
```

3.4 Teste Caseiro do Binário ARM com QEMU (*User Mode*)

Você pode verificar se a compilação cruzada foi bem-sucedida sem precisar esperar pelo laboratório. O emulador QEMU permite rodar binários de outras arquiteturas no seu próprio PC Linux:

```
# 1. Instalar o suporte ao QEMU User Space na sua maquina Linux
sudo apt update && sudo apt install -y qemu-user qemu-user-static

# 2. Executar o binario ARM apontando para o sysroot da toolchain
qemu-arm -L /caminho/da/toolchain/arm-buildroot-linux-gnueabihf/sysroot ./
    app_sensor_arm /tmp/mock_iio/in_voltage0_raw
```

Se o binário rodar sem falhas no QEMU, a execução física na placa STM32MP157D-DK1 funcionará de primeira.

3.5 Procedimento de Bancada: Rede, Transferência e Execução Remota

No laboratório, a comunicação entre o computador e a placa é realizada através de um cabo de rede Ethernet comum (RJ45):

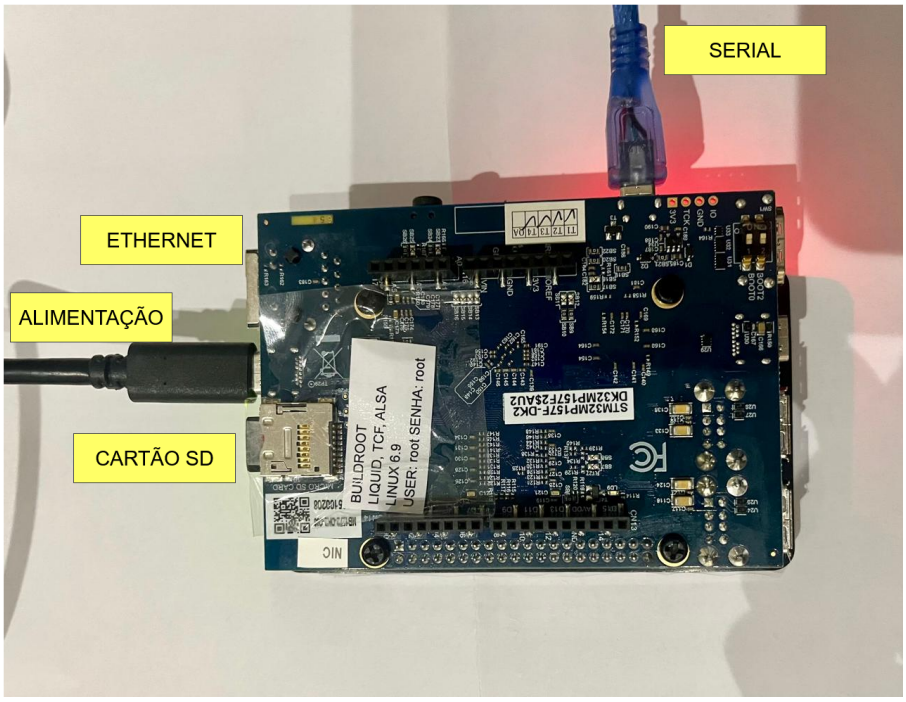


Figura 2: Porta Ethernet da placa STM32MP157D-DK1 (ao lado da entrada USB-C de alimentação).

Roteiro de Bancada para a Entrega 2

1. **Conexão Física e Alimentação:**
   - Conecte o cabo USB-C na porta `PWR_IN`.
   - Conecte o cabo de rede Ethernet entre a placa e a porta de rede do computador.
   - Certifique-se de que os 3 fios do sensor HW-201 permanecem conectados nos pinos validados na Entrega 0 (Pino 4 = 3V3, Pino 7 = GND, Pino 5 = ADC).
2. **Configuração da Rede Local no PC:**
   - A placa STM32MP157D-DK1 possui o endereço IP fixo configurado no laboratório: `192.168.42.2`.
   - No seu computador, abra as configurações de rede da interface Ethernet e defina um IP estático na mesma sub-rede, por exemplo:
     - IP do Computador: `192.168.42.10`
     - Máscara de Sub-rede: `255.255.255.0`
     - Gateway: *Pode deixar em branco.*
3. **Transferir o Binário via SCP (*Secure Copy*):**
   No terminal do seu computador (onde você gerou o arquivo `app_sensor_arm`), envie o programa para a pasta `/home/root` da placa:

   ```
   scp app_sensor_arm root@192.168.42.2:/home/root/
   ```

   Quando solicitar a senha, digite `root`.
4. **Acessar a Placa via SSH e Executar o Programa:**

   ```
   # Acessa o shell Linux da placa remotamente via rede
   ssh root@192.168.42.2

   # Entra na pasta e concede permissao de execucao
   ```

```
cd /home/root
chmod +x app_sensor_arm

# Executa o programa lendo o sensor fisico real
./app_sensor_arm
```

3.6 Calibração Prática do Sensor HW-201

Com o programa rodando e exibindo as leituras em tempo real no terminal da placa:

- **Ajuste do Trimpot:** Utilize uma chave de fenda pequena no potenciômetro quadrado azul do sensor HW-201. Gire até que o LED verde do sensor acenda exatamente quando a tampa do baú/obstáculo estiver na distância planejada (entre 5 cm e 15 cm).
- **Registro de Evidências (Prints):**
  - Tire um print do terminal mostrando as leituras contínuas com a **tampa fechada / compartimento seguro** (valores baixos de ADC).
  - Abra o compartimento e tire um print imediato mostrando a comutação para **alarme de violação** (valores altos de ADC).

3.7 Requisitos do `README.md` para a Entrega 2

Conforme a Instrução Geral nº 5 do edital, o arquivo `README.md` deve ser atualizado com:

1. Instruções exatas e reproduzíveis do comando de compilação cruzada.
2. Comando de transferência via `scp` e de execução via `ssh`.
3. Imagens/prints do terminal anexados na pasta `docs/` mostrando a aplicação em funcionamento real.
4. Submissão do link do commit final da Entrega 2 no Google Classroom até **02/11/2026 às 23:59**.

4 MÓDULO 3: Visão Geral das Etapas Subsequentes

Para que o desenvolvimento do grupo mantenha coerência arquitetural desde os Passos 1 e 2, este é o horizonte das fases seguintes:

Etapa	Conceitos-Chave e Desafios de Engenharia
Passo 3 (09/11):	**Telemetria via Socket UDP em C++ (POSIX):** Criação de uma classe modular (ex.: `UdpTransmitter`) utilizando as bibliotecas de sistema `<sys/socket.h>` e `<arpa/inet.h>`. O programa embarcado na STM32MP1 enviará datagramas contínuos contendo identificador do sensor, estado da carga e timestamp para a porta UDP do computador central.
Passo 4 (16/11):	**Interface Gráfica de Monitoramento no Computador (GUI):** Desenvolvimento de dashboard no PC em Python (PyQt / Matplotlib / PyQtGraph) ou C++/Qt. A interface escuta na porta UDP, plota gráfico deslizante de 30 a 60 segundos com taxa de atualização fluida, exibe alerta visual de invasão e exporta logs em arquivo CSV.
Passo 5 (16/11):	**Documentação Consolidada, Vídeo e Simulação Integrada:** Produção do vídeo demonstrativo mostrando o hardware operando de ponta a ponta (atuação física no sensor → reação imediata na tela do PC), refinamento do diagrama UML no `README.md` e participação no teste conjunto com os 11 sensores da turma.